

KELAS INDUSTRI	MODUL PEMBELAJARAN KELAS INDUSTRI SERVER ACTIONS, CRUD PENUH & PROTEKSI HALAMAN ADMIN	Kelas XI Pertemuan 04
-----------------------	--	----------------------------------

Satuan Pendidikan	:	Sekolah Menengah Kejuruan (SMK)
Kompetensi Keahlian	:	Rekayasa Perangkat Lunak (RPL / PPLG)
Kelas / Semester	:	XI (Sebelas) / Semester Ganjil
Pertemuan Ke	:	04 (Empat) dari Seri Next.js Dasar sampai Mahir
Alokasi Waktu	:	4 JP (4 x 45 Menit = 180 Menit)
Topik Pertemuan	:	Server Actions Next.js 15 untuk Operasi CRUD Penuh (Create, Update, Delete) dan Proteksi Halaman Admin Menggunakan Supabase Auth
Pola Pembelajaran	:	100% Praktik Kejuruan Project-Based Learning Teori 30% : Praktik Terbimbing 70%

I. CAPAIAN PEMBELAJARAN (LEARNING OUTCOMES)

1. Siswa memahami apa itu Server Actions Next.js 15 dan perbedaannya dengan Route Handler yang sudah diperkenalkan di akhir Pertemuan 03.
2. Siswa memahami mengapa Server Actions lebih direkomendasikan untuk mutasi data (Create, Update, Delete) dibanding Route Handler dalam ekosistem Next.js App Router modern.
3. Siswa mampu membuat Server Action untuk operasi Create menambahkan data proyek baru ke tabel Supabase langsung dari form di halaman admin.
4. Siswa mampu membuat Server Action untuk operasi Update mengubah data proyek yang sudah ada di tabel Supabase melalui form edit.
5. Siswa mampu membuat Server Action untuk operasi Delete menghapus baris data proyek dari tabel Supabase dengan konfirmasi tombol di halaman admin.
6. Siswa memahami konsep dasar Supabase Auth (autentikasi email/password) dan perbedaan antara pengguna yang sudah login dan pengguna anonim.
7. Siswa mampu membuat halaman login admin sederhana yang memeriksa sesi login Supabase sebelum menampilkan konten admin.
8. Siswa mampu memproteksi route halaman admin menggunakan middleware Next.js, sehingga pengunjung yang belum login dialihkan ke halaman login.
9. Siswa memahami alur CRUD penuh: data ditambah/diubah/dihapus lewat halaman admin yang terkunci, lalu perubahan langsung terlihat di halaman portfolio publik yang sudah dibangun sejak Pertemuan 01-03.
10. Siswa mampu membedakan halaman publik (portfolio yang bisa diakses siapa saja) dengan halaman admin yang hanya bisa diakses setelah login.

II. KONTEKS INDUSTRI & RELEVANSI KARIR

Pada Pertemuan 03, kalian sudah berhasil memindahkan data proyek dari file kode statis ke database Supabase dan menampilkannya di halaman portfolio. Namun satu masalah masih tersisa: untuk menambah, mengubah, atau menghapus data, kalian masih harus membuka dashboard Supabase secara manual. Di dunia nyata, pemilik website misalnya seorang mahasiswa yang sedang melamar kerja atau seorang freelancer yang ingin update portofolionya tidak selalu paham cara menggunakan dashboard Supabase. Mereka membutuhkan halaman admin yang sederhana dan terkunci, langsung di dalam website itu sendiri. Konsep halaman admin yang terproteksi oleh login ini adalah fitur standar di hampir setiap aplikasi web profesional, mulai dari sistem manajemen konten (CMS), e-commerce, hingga aplikasi internal perusahaan. Server Actions adalah cara modern dan resmi Next.js 15 untuk menangani operasi menulis data (mutasi) dari sisi server secara langsung dari komponen React, tanpa harus membuat file API terpisah. Kemampuan ini membangun halaman CRUD admin yang terproteksi adalah keterampilan yang membedakan developer yang hanya bisa menampilkan data dengan developer yang bisa membangun aplikasi web yang lengkap dan siap digunakan oleh orang nyata.

Platform & Referensi Belajar Standar Industri

Platform	URL / Tautan	Deskripsi & Kegunaan
Dokumentasi Resmi Next.js - Server Actions dan Mutations	https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations	Panduan resmi memahami dan menggunakan Server Actions untuk operasi mutasi data di Next.js App Router
Dokumentasi Resmi Supabase - Auth Email/Password	https://supabase.com/docs/guides/auth/passwords	Panduan resmi membuat sistem login email dan password menggunakan Supabase Auth
Dokumentasi Resmi Next.js - Middleware	https://nextjs.org/docs/app/building-your-application/routing/middleware	Panduan resmi menggunakan middleware Next.js untuk memproteksi route tertentu dari akses tanpa autentikasi
Dokumentasi Resmi Supabase - Auth Helpers untuk Next.js (SSR)	https://supabase.com/docs/guides/auth/server-side/nextjs	Panduan resmi menggunakan Supabase Auth di sisi server Next.js agar sesi login bisa dibaca di Server Component dan middleware

III. KEBUTUHAN PERANGKAT & TOOLS

No.	Kebutuhan	Detail	Keterangan
1	Project Profil Siswa dari Pertemuan 01-03	Folder project profil-siswa yang sudah terhubung ke Supabase, memiliki tabel proyek dengan data, lib/supabase.ts, dan halaman / proyek yang menampilkan data dari database.	Wajib - fondasi kelanjutan dari semua pertemuan sebelumnya
2	Project Supabase Aktif dari Pertemuan 03	Project Supabase yang sudah dibuat dan dikonfigurasi pada Pertemuan 03, termasuk tabel proyek, policy READ publik, dan kredensial di .env.local.	Wajib - sudah tersedia dari pertemuan sebelumnya
3	Package @supabase/ssr (Baru Diinstal Pertemuan Ini)	Package tambahan dari Supabase khusus untuk menangani sesi autentikasi di sisi server Next.js (SSR). Berbeda dari @supabase/supabase-js yang sudah dipakai di Pertemuan 03.	Wajib - akan diinstal di awal praktik

No.	Kebutuhan	Detail	Keterangan
4	Koneksi Internet Stabil	Semua operasi CRUD dan autentikasi melibatkan komunikasi dengan server Supabase di cloud.	Wajib - sama seperti Pertemuan 03
5	Visual Studio Code	Editor kode dengan terminal terintegrasi untuk menjalankan perintah instalasi dan dev server.	Direkomendasikan

IV. PROJECT PERTEMUAN INI

Nama Project	Dashboard Admin Terproteksi untuk Website Profil Siswa & Portfolio
Teknologi	Next.js 15 + TypeScript + Tailwind CSS + Supabase (PostgreSQL + supabase-js + @supabase/ssr + Supabase Auth)
Tingkat Kesulitan	Menengah-Atas Kejuruan (Fase F SMK RPL / PPLG)
Output Akhir	Website Profil Siswa & Portfolio dengan halaman admin terproteksi di /admin/proyek yang memungkinkan pemilik website menambah, mengedit, dan menghapus data proyek langsung dari browser tanpa membuka dashboard Supabase, serta halaman login di /admin/login yang memproteksi seluruh route /admin/*.
Roadmap Lanjutan	Pertemuan 05 akan melakukan deployment ke Vercel secara nyata dengan environment variable produksi, serta mengoptimalkan metadata SEO dan performa website portfolio yang sudah lengkap ini.

Melanjutkan project Website Profil Siswa & Portfolio dari Pertemuan 01-03. Pertemuan ini menambahkan halaman admin di route /admin yang terkunci dengan login Supabase Auth (email/password). Di dalam halaman admin tersebut, siswa membuat antarmuka untuk menambah proyek baru, mengedit proyek yang ada, dan menghapus proyek semua menggunakan Server Actions Next.js 15 yang terhubung langsung ke tabel proyek Supabase. Halaman portfolio publik (/proyek) yang sudah ada tetap tidak berubah tampilannya, hanya kini datanya bisa dikelola oleh admin yang sudah login.

V. ALUR PEMBELAJARAN 2 JP (90 MENIT)

No.	Durasi	Mode / Format	Aktivitas Pembelajaran
1	15 Menit	Instruktur - Refleksi & Pemantik Masalah	Segmen 1: Pembukaan & Refleksi CRUD (15 Menit) - Instruktur membuka kelas dan mengajak siswa membuka dashboard Supabase mereka, lalu mencoba menambah satu baris data baru langsung dari Table Editor. - Ajukan pertanyaan pemantik: 'Kalau pemilik website ini bukan kalian, tapi adik kalian yang tidak paham Supabase, bagaimana caranya dia bisa menambah proyek baru ke portfolionya sendiri?' - Jelaskan roadmap hari ini: 'Kita akan membuat halaman admin di dalam website itu sendiri, yang bisa dibuka di browser, tapi terkunci dengan password. Admin bisa tambah, edit, dan hapus data langsung dari sana.' - Perkenalkan dua konsep baru hari ini: Server Actions (cara baru Next.js untuk menyimpan data) dan Supabase Auth (sistem login).
2	30 Menit	Instruktur - Penjelasan Konsep & Demo	Segmen 2: Teori 1-2 Server Actions & Konsep Auth (30 Menit) - Jelaskan apa itu Server Actions dan bedanya dengan Route Handler yang sudah dikenalkan di Step 12 Pertemuan 03. - Demo singkat: tunjukkan bagaimana direktif 'use server' membuat sebuah fungsi berjalan di sisi server, bukan di browser. - Jelaskan konsep autentikasi sederhana: siapa yang bisa login, bagaimana sesi disimpan, dan kenapa halaman admin harus dijaga. - Perkenalkan alur proteksi route menggunakan middleware Next.js: 'Sebelum halaman /admin dibuka, middleware mengecek dulu: sudah login belum?'
3	40 Menit	Siswa Praktik Mandiri + Asistensi Instruktur	Segmen 3: Praktik Terbimbing 1 Setup Auth & Halaman Login (40 Menit) - Step 1-2: Siswa menginstal package <code>@supabase/ssr</code> dan membuat dua Supabase client baru (satu untuk server, satu untuk browser). - Step 3: Siswa membuat akun admin pertama di dashboard Supabase Authentication. - Step 4: Siswa menambahkan tiga policy baru (INSERT/UPDATE/DELETE authenticated) ke tabel proyek di Supabase. - Step 5: Siswa membuat middleware.ts di root project untuk memproteksi seluruh route <code>/admin/*</code>. - Step 6: Siswa membuat halaman login di <code>app/admin/login/page.tsx</code> dengan form email dan password menggunakan Server Action. - Instruktur berkeliling memastikan setiap siswa berhasil login dan diarahkan ke halaman <code>/admin</code>.

No.	Durasi	Mode / Format	Aktivitas Pembelajaran
4	75 Menit	Siswa Coding Mandiri + Asistensi Instruktur	Segmen 4: Praktik Terbimbing 2 Dashboard Admin & Operasi CRUD (75 Menit) • Step 7: Siswa membuat layout halaman admin (app/admin/layout.tsx) dengan navigasi dan tombol logout. • Step 8: Siswa membuat halaman dashboard utama admin (app/admin/proyek/page.tsx) yang menampilkan semua data proyek dalam bentuk tabel, beserta form tambah proyek baru menggunakan Server Action Create. • Step 9: Siswa membuat halaman edit proyek dan Server Action Update form yang sudah terisi data lama dan menyimpan perubahan ke Supabase. • Step 10: Siswa membuat halaman konfirmasi hapus dan Server Action Delete tombol hapus dengan halaman konfirmasi sebelum benar-benar menghapus. • Instruktur berkeliling memastikan setiap siswa berhasil mencoba seluruh alur CRUD dari halaman admin, lalu memverifikasi perubahan di halaman /proyek publik.
5	20 Menit	Praktik Mandiri & Evaluasi Instruktur	Segmen 5: Uji Coba Menyeluruh & Penutup (20 Menit) • Seluruh siswa menguji alur penuh: buka / admin/login, login, tambah proyek baru, cek halaman /proyek, edit proyek tadi, hapus proyek, lalu logout. • Siswa mencoba membuka /admin/proyek tanpa login (di jendela incognito) untuk memverifikasi bahwa middleware berhasil memblokir akses. • Pemberian arahan Tugas Mandiri untuk diselesaikan di rumah. • Preview Pertemuan 05: 'Minggu depan project kalian yang sudah lengkap ini akan kita deploy ke internet menggunakan Vercel, sehingga bisa diakses siapa saja dari seluruh dunia!'

VI. MATERI TEORI & KONSEP KUNCI

1. Apa itu Server Actions dan Kenapa Lebih Baik dari Route Handler?

Pengantar: Di akhir Pertemuan 03 (Step 12 opsional), kalian sudah pernah berkenalan dengan Route Handler di file `app/api/kontak/route.ts` untuk menerima data dari form dan menyimpannya ke Supabase. Route Handler berfungsi dengan baik, tapi membutuhkan dua bagian terpisah: file API-nya sendiri, dan kode di sisi browser untuk mengirim `fetch()` ke file API itu.

Server Actions adalah cara baru yang lebih ringkas: kalian bisa menulis fungsi server langsung di dalam komponen React, ditandai dengan direktif `'use server'` di baris pertama fungsi (atau file-nya). Next.js kemudian secara otomatis menangani komunikasi antara browser dan server tanpa kalian perlu menulis `fetch()` secara manual.

Perbedaan utama Server Actions vs Route Handler:

- Route Handler: Harus membuat file terpisah di `app/api/...`, browser harus `fetch()` manual ke URL file tersebut, lebih fleksibel untuk API publik yang perlu diakses dari luar website.
- Server Actions: Fungsi biasa dengan `'use server'`, bisa langsung dipakai sebagai action di form HTML atau dipanggil dari Client Component, tidak butuh URL API terpisah, lebih disarankan untuk operasi mutasi data di dalam aplikasi Next.js itu sendiri.

Analogi: Route Handler itu seperti kasir di konter berbeda yang harus kalian datang secara terpisah. Server Actions itu seperti sistem self-checkout modern kalian langsung berinteraksi dengan mesin yang langsung terhubung ke sistem, tanpa perlu ke konter terpisah.

Kapan pakai mana: Untuk kebutuhan materi ini (form di halaman admin yang menyimpan data), Server Actions adalah pilihan yang tepat. Route Handler lebih cocok kalau kalian ingin membuat API yang bisa dipanggil dari aplikasi lain (misalnya aplikasi mobile).

2. Konsep Autentikasi: Siapa yang Boleh Masuk ke Halaman Admin?

Autentikasi (authentication) adalah proses memverifikasi identitas seseorang: 'Apakah kamu benar-benar orang yang kamu klaim?' Ini berbeda dengan otorisasi (authorization): 'Apakah orang yang sudah diverifikasi ini punya izin untuk melakukan aksi tertentu?'

Untuk halaman admin portfolio ini, konsepnya sederhana:

- Publik (belum login): Bisa melihat halaman `/proyek`, `/tentang`, dan halaman portfolio lainnya. TIDAK bisa membuka `/admin`.
- Admin (sudah login): Bisa membuka `/admin`, melihat semua data proyek, dan melakukan operasi tambah/edit/hapus.

Supabase Auth menyediakan sistem login email dan password siap pakai. Saat pengguna berhasil login, Supabase membuat sebuah sesi (session) yang disimpan di browser dalam bentuk cookie. Setiap kali pengguna membuka halaman berikutnya, cookie itu dikirim ke server Next.js, yang kemudian bertanya ke Supabase: 'Apakah sesi ini masih valid?' Kalau ya, pengguna diperbolehkan masuk. Kalau tidak (belum login, atau sesi sudah kadaluarsa), pengguna diarahkan kembali ke halaman login.

PENTING Satu akun admin saja: Untuk portfolio pribadi siswa, kalian TIDAK perlu sistem registrasi pengguna yang bisa didaftarkan siapa saja. Cukup buat satu akun admin langsung di dashboard Supabase Authentication, dan hanya akun itu yang bisa login ke halaman /admin. Ini jauh lebih sederhana dan aman untuk skala portfolio pribadi.

3. Middleware Next.js: Penjaga Pintu Sebelum Halaman Dibuka

Middleware adalah kode yang berjalan di Next.js SEBELUM sebuah halaman atau route diproses. Bayangkan middleware seperti satpam di lobi gedung: sebelum kamu naik ke lantai yang kamu tuju, satpam memeriksa ID kamu terlebih dahulu. Kalau ID-nya valid, kamu dipersilakan naik. Kalau tidak, kamu diarahkan ke konter pendaftaran tamu.

Dalam konteks halaman admin portfolio:

1. Pengguna mengetik /admin/proyek di browser.
2. SEBELUM Next.js merender halaman itu, middleware.ts berjalan lebih dulu.
3. Middleware memeriksa: apakah ada cookie sesi Supabase yang valid di browser pengguna?
4. Kalau ya: pengguna diperbolehkan masuk ke /admin/proyek.
5. Kalau tidak: pengguna secara otomatis diarahkan (redirect) ke /admin/login.

Keuntungan menggunakan middleware (bukan mengecek login di dalam setiap halaman satu per satu): Kalian cukup menulis logika pemeriksaan di SATU file middleware.ts, dan semua route /admin/* otomatis terlindungi, termasuk halaman-halaman admin baru yang mungkin kalian tambahkan nanti.

File middleware.ts harus diletakkan di ROOT folder project (sejajar dengan folder app/, bukan di dalam app/), agar Next.js otomatis mengenalinya.

4. Dua Jenis Supabase Client untuk Next.js dengan Auth

Di Pertemuan 03, kalian membuat satu file lib/supabase.ts yang berisi satu Supabase client menggunakan createClient dari @supabase/supabase-js. Client ini cukup untuk operasi READ publik karena menggunakan anon key dan tidak perlu mengelola sesi login.

Untuk menangani autentikasi di Next.js App Router, kita perlu package tambahan bernama @supabase/ssr (SSR = Server-Side Rendering). Package ini menyediakan dua jenis client:

1. Server Client (untuk Server Component dan Server Action): Dibuat dengan createServerClient dari @supabase/ssr. Membaca cookie dari request HTTP untuk mengetahui siapa yang sedang login. Dipakai di Server Component, Server Actions, dan middleware.
2. Browser Client (untuk Client Component): Dibuat dengan createBrowserClient dari @supabase/ssr. Menyimpan dan membaca cookie langsung dari browser. Dipakai di Client Component yang perlu tahu status login di sisi browser.

Di praktik nanti, kalian akan membuat dua file helper terpisah (lib/supabase-server.ts dan lib/supabase-browser.ts) yang masing-masing berisi satu jenis client, sehingga di halaman atau komponen manapun kalian tinggal import file yang sesuai.

FILE lib/supabase.ts dari Pertemuan 03 TETAP DIPERTAHANKAN untuk operasi READ publik di halaman portfolio yang sudah ada. Tidak perlu diubah.

5. Operasi CRUD Penuh dengan Server Actions: Gambaran Umum

Pada Pertemuan 03 kalian sudah belajar Read. Pertemuan ini melengkapi tiga operasi sisanya: Create, Update, dan Delete. Berikut gambaran masing-masing:

Create (Tambah Data Baru):

- Pengguna mengisi form tambah proyek di halaman admin.
- Saat form di-submit, Server Action dipanggil dengan data dari form.
- Server Action menyimpan data ke Supabase menggunakan `.insert()`.
- Halaman admin di-refresh untuk menampilkan data terbaru (menggunakan `revalidatePath`).

Update (Edit Data yang Sudah Ada):

- Pengguna mengklik tombol Edit pada salah satu proyek di tabel admin.
- Halaman edit dibuka dengan form yang sudah terisi data proyek tersebut.
- Setelah diubah dan disimpan, Server Action memperbarui baris yang tepat di Supabase menggunakan `.update().eq('id', ...)`.
- `revalidatePath` dipanggil agar halaman `/proyek publik` juga menampilkan data terbaru.

Delete (Hapus Data):

- Pengguna mengklik tombol Hapus pada salah satu proyek.
- Halaman konfirmasi ditampilkan agar tidak terhapus secara tidak sengaja.
- Server Action menghapus baris tersebut dari Supabase menggunakan `.delete().eq('id', ...)`.
- Halaman admin di-refresh, data yang dihapus tidak lagi muncul.

Konsep penting baru `revalidatePath`: Next.js secara default menyimpan cache halaman untuk performa. Setiap kali data diubah lewat Server Action, kalian perlu memanggil `revalidatePath('/proyek')` dan `revalidatePath('/admin/proyek')` agar Next.js menghapus cache lama dan menampilkan data yang benar-benar terbaru.

6. Keamanan: Policy Supabase untuk Operasi Tulis dari Server

Di Pertemuan 03, kalian membuat policy `SELECT` (baca) yang bersifat publik untuk tabel proyek, sehingga siapa saja bisa membaca data proyek. Policy ini tetap dibiarkan seperti itu untuk halaman portfolio publik.

Untuk operasi `INSERT`, `UPDATE`, dan `DELETE`, kita perlu menambahkan policy baru. Tapi hati-hati: policy ini TIDAK boleh bersifat publik, karena kalau dibuka untuk umum, siapapun (termasuk peretas) bisa menambah, mengubah, atau menghapus data proyek kalian.

Solusi yang benar: Policy `INSERT/UPDATE/DELETE` hanya mengizinkan pengguna yang sudah terautentikasi (`authenticated`). Ini artinya: Supabase hanya akan menjalankan operasi write kalau

permintaannya datang dengan sesi login yang valid. Tanpa login yang valid, Supabase akan menolak operasi tersebut, bahkan kalau seseorang berhasil memanggil Server Action kalian secara langsung.

Ringkasan policy yang perlu ada di tabel proyek setelah Pertemuan ini:

- SELECT (baca): publik siapa saja boleh membaca (sudah dibuat di Pertemuan 03).
- INSERT (tambah): authenticated hanya user yang sudah login.
- UPDATE (edit): authenticated hanya user yang sudah login.
- DELETE (hapus): authenticated hanya user yang sudah login.

Ini adalah contoh nyata konsep pertahanan berlapis dalam keamanan aplikasi: middleware memblokir akses ke halaman admin, DAN policy Supabase memblokir operasi tulis tanpa login yang valid.

VII. PANDUAN PRAKTIK STEP-BY-STEP (HANDS-ON LAB)

Step 1 - Instalasi Package @supabase/ssr

Buka terminal VS Code di folder project profil-siswa yang sudah ada, lalu instal package baru. Package ini dibutuhkan agar Supabase bisa membaca dan menyimpan sesi login di cookie Next.js, yang tidak bisa dilakukan oleh @supabase/supabase-js sendirian.

```
cd profil-siswa
npm install @supabase/ssr
```

Step 2 - Membuat Dua Helper Client Supabase untuk Auth

Buat dua file helper baru di dalam folder lib/. File lib/supabase.ts dari Pertemuan 03 TIDAK diubah. Dua file baru ini khusus untuk menangani sesi autentikasi. Buat file pertama: lib/supabase-server.ts, lalu buat file kedua: lib/supabase-browser.ts.

```
// lib/supabase-server.ts
// Dipakai di Server Component, Server Action, dan Middleware
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

export async function createSupabaseServerClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
```

```

        cookieStore.set(name, value, options)
    );
} catch {
    // Diabaikan jika dipanggil dari Server Component yang tidak bisa menulis
    cookie
}
},
},
}
);
}

```

Step 3 - Membuat Akun Admin di Supabase Authentication

Buka dashboard Supabase project kalian, masuk ke menu Authentication di sidebar kiri. Klik tab Users, lalu klik tombol Invite user atau Add user. Isi email (boleh email kalian sendiri atau email sekolah) dan password yang kuat. Akun ini adalah satu-satunya akun yang bisa login ke halaman admin portfolio kalian. CATAT email dan password tersebut karena akan dipakai untuk testing.

Step 4 - Menambahkan Policy CRUD untuk Tabel proyek

Buka menu Authentication > Policies di dashboard Supabase (atau klik ikon shield di Table Editor pada tabel proyek). Saat ini sudah ada satu policy SELECT publik dari Pertemuan 03 JANGAN dihapus. Tambahkan tiga policy baru dengan mengklik New Policy untuk setiap policy berikut:

Policy 1: INSERT hanya untuk user yang sudah login

Klik 'New Policy', pilih template 'Enable insert for authenticated users only'

Konfigurasi:

Policy name : Allow authenticated insert

Operation : INSERT

Target roles: authenticated

Policy 2: UPDATE hanya untuk user yang sudah login

Klik 'New Policy', pilih template 'Enable update for authenticated users only'

Konfigurasi:

Policy name : Allow authenticated update

Operation : UPDATE

Target roles: authenticated

Policy 3: DELETE hanya untuk user yang sudah login

Klik 'New Policy', pilih template 'Enable delete for authenticated users only'

Konfigurasi:

Policy name : Allow authenticated delete

Operation : DELETE

Target roles: authenticated

Step 5 - Membuat Middleware untuk Proteksi Route /admin

Buat file baru bernama middleware.ts di ROOT folder project (sejajar dengan package.json dan folder app/, BUKAN di dalam app/). File ini akan berjalan otomatis sebelum setiap request dan memeriksa sesi login.

```
// middleware.ts letakkan di root project, sejajar dengan package.json
import { createServerClient } from '@supabase/ssr';
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export async function middleware(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          );
          supabaseResponse = NextResponse.next({ request });
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  const { data: { user } } = await supabase.auth.getUser();

  const isAdminRoute = request.nextUrl.pathname.startsWith('/admin');
  const isLoginPage = request.nextUrl.pathname === '/admin/login';

  if (isAdminRoute && !isLoginPage && !user) {
    return NextResponse.redirect(new URL('/admin/login', request.url));
  }

  if (isLoginPage && user) {
    return NextResponse.redirect(new URL('/admin/proyek', request.url));
  }
}
```

```

    return supabaseResponse;
  }

  export const config = {
    matcher: ['/admin/:path*'],
  };

```

Step 6 - Membuat Halaman Login Admin (/admin/login)

Buat folder `app/admin/login/`, lalu buat file `page.tsx` di dalamnya. Halaman ini berisi form email dan password. Server Action di dalam file ini yang menangani proses login menggunakan Supabase Auth.

```

// app/admin/login/page.tsx
import { redirect } from 'next/navigation';
import { createSupabaseServerClient } from '@lib/supabase-server';

async function loginAction(formData: FormData) {
  'use server';

  const email = formData.get('email') as string;
  const password = formData.get('password') as string;

  const supabase = await createSupabaseServerClient();
  const { error } = await supabase.auth.signInWithPassword({ email, password });

  if (error) {
    redirect('/admin/login?error=Kredensial+tidak+valid');
  }

  redirect('/admin/proyek');
}

export default async function AdminLoginPage({
  searchParams,
}: {
  searchParams: Promise<{ error?: string }>;
}) {
  const params = await searchParams;

  return (
    <main className="min-h-screen flex items-center justify-center bg-slate-100">
      <div className="bg-white p-8 rounded-2xl shadow-lg w-full max-w-sm">
        <h1 className="text-2xl font-bold text-slate-800 mb-2">Admin Login</h1>
        <p className="text-slate-500 text-sm mb-6">Masuk untuk mengelola data
        portfolio</p>

```

```

    {params.error && (
      <p className="text-red-600 text-sm mb-4 bg-red-50 p-3 rounded-lg">
        {params.error}
      </p>
    )}

    <form action={loginAction} className="space-y-4">
      <div>
        <label htmlFor="email" className="block text-sm font-medium text-slate-700 mb-1">Email</label>
        <input id="email" name="email" type="email" required
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
      </div>
      <div>
        <label htmlFor="password" className="block text-sm font-medium text-slate-700 mb-1">Password</label>
        <input id="password" name="password" type="password" required
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
      </div>
      <button type="submit"
        className="w-full bg-blue-600 text-white py-2 rounded-lg font-medium hover:bg-blue-700 transition-colors">
        Masuk
      </button>
    </form>
  </div>
</main>
);
}

```

Step 7 - Membuat Layout Admin & Tombol Logout

Buat file `app/admin/layout.tsx`. File ini adalah layout khusus untuk semua halaman di bawah route / admin, berisi navigasi header admin dan tombol logout. Server Action logout juga ditempatkan di sini.

```

// app/admin/layout.tsx
import { redirect } from 'next/navigation';
import { createSupabaseServerClient } from '@/lib/supabase-server';
import Link from 'next/link';

async function logoutAction() {
  'use server';
  const supabase = await createSupabaseServerClient();
  await supabase.auth.signOut();
}

```

```

    redirect('/admin/login');
  }

export default async function AdminLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  const supabase = await createSupabaseServerClient();
  const { data: { user } } = await supabase.auth.getUser();

  return (
    <div className="min-h-screen bg-slate-50">
      <header className="bg-white border-b border-slate-200 px-6 py-4 flex items-center justify-between">
        <div className="flex items-center gap-6">
          <span className="font-bold text-slate-800">Admin Panel</span>
          <nav className="flex gap-4">
            <Link href="/admin/proyek"
              className="text-sm text-slate-600 hover:text-blue-600 transition-colors">
              Manajemen Proyek
            </Link>
          </nav>
        </div>
        <div className="flex items-center gap-4">
          <span className="text-sm text-slate-500">{user?.email}</span>
          <form action={logoutAction}>
            <button type="submit"
              className="text-sm bg-slate-100 hover:bg-slate-200 text-slate-700 px-4 py-2 rounded-lg transition-colors">
              Logout
            </button>
          </form>
        </div>
      </header>
      <main className="max-w-5xl mx-auto px-6 py-8">{children}</main>
    </div>
  );
}

```

Step 8 - Membuat Dashboard Admin & Server Action Create

Buat folder `app/admin/proyek/`, lalu buat file `page.tsx`. Halaman ini menampilkan semua data proyek dalam tabel dan menyediakan form untuk menambahkan proyek baru menggunakan Server Action.

```
// app/admin/proyek/page.tsx
import { revalidatePath } from 'next/cache';
import { createSupabaseServerClient } from '@lib/supabase-server';
import Link from 'next/link';

async function tambahProyekAction(formData: FormData) {
  'use server';

  const supabase = await createSupabaseServerClient();

  const { error } = await supabase.from('proyek').insert({
    judul: formData.get('judul') as string,
    deskripsi: formData.get('deskripsi') as string,
    teknologi: formData.get('teknologi') as string,
    link: (formData.get('link') as string) || null,
  });

  if (error) {
    console.error('Gagal menambah proyek:', error.message);
  }

  revalidatePath('/admin/proyek');
  revalidatePath('/proyek');
}

export default async function AdminProyekPage() {
  const supabase = await createSupabaseServerClient();
  const { data: daftarProyek } = await supabase
    .from('proyek')
    .select('*')
    .order('id', { ascending: true });

  return (
    <div className="space-y-8">
      <h1 className="text-2xl font-bold text-slate-800">Manajemen Proyek</h1>

      <div className="bg-white rounded-2xl border border-slate-200 overflow-hidden">
        <table className="w-full text-sm">
          <thead className="bg-slate-50 border-b border-slate-200">
            <tr>
              <th className="text-left px-4 py-3 font-medium text-slate-600">Judul</th>
              <th className="text-left px-4 py-3 font-medium text-slate-600">Teknologi</th>
              <th className="text-left px-4 py-3 font-medium text-slate-600">Aksi</th>
            </tr>

```

```

</thead>
<tbody className="divide-y divide-slate-100">
  {daftarProyek?.map((proyek) => (
    <tr key={proyek.id} className="hover:bg-slate-50">
      <td className="px-4 py-3 text-slate-800 font-medium">{proyek.judul}</td>
      <td className="px-4 py-3 text-slate-500">{proyek.teknologi}</td>
      <td className="px-4 py-3">
        <div className="flex gap-2">
          <Link href={'/admin/proyek/edit/' + proyek.id}
            className="text-xs bg-blue-50 text-blue-700 hover:bg-blue-100
px-3 py-1.5 rounded-lg transition-colors">
            Edit
          </Link>
          <Link href={'/admin/proyek/hapus/' + proyek.id}
            className="text-xs bg-red-50 text-red-700 hover:bg-red-100 px-3
py-1.5 rounded-lg transition-colors">
            Hapus
          </Link>
        </div>
      </td>
    </tr>
  )})
</tbody>
</table>
</div>

<div className="bg-white rounded-2xl border border-slate-200 p-6">
  <h2 className="text-lg font-semibold text-slate-800 mb-4">Tambah Proyek Baru</h2>
  <form action={tambahProyekAction} className="space-y-4">
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div>
        <label htmlFor="judul" className="block text-sm font-medium text-slate-700 mb-1">Judul Proyek</label>
        <input id="judul" name="judul" required
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm
focus:outline-none focus:ring-2 focus:ring-blue-500" />
      </div>
      <div>
        <label htmlFor="teknologi" className="block text-sm font-medium text-slate-700 mb-1">Teknologi (pisah koma)</label>
        <input id="teknologi" name="teknologi"
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm
focus:outline-none focus:ring-2 focus:ring-blue-500" />
      </div>
    </div>
  </form>

```

```

        </div>
      </div>
      <div>
        <label htmlFor="deskripsi" className="block text-sm font-medium text-slate-700 mb-1">Deskripsi</label>
        <textarea id="deskripsi" name="deskripsi" rows={3}
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
        </div>
      <div>
        <label htmlFor="link" className="block text-sm font-medium text-slate-700 mb-1">Link Proyek (opsional)</label>
        <input id="link" name="link" type="url"
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
        </div>
        <button type="submit"
          className="bg-blue-600 text-white px-5 py-2 rounded-lg text-sm font-medium hover:bg-blue-700 transition-colors">
          Simpan Proyek
        </button>
      </form>
    </div>
  </div>
);
}

```

Step 9 - Membuat Halaman Edit & Server Action Update

Buat folder `app/admin/proyek/edit/[id]/`, lalu buat file `page.tsx`. Halaman ini mengambil data proyek berdasarkan id dari URL, lalu menampilkan form yang sudah terisi data tersebut. Server Action di dalamnya menyimpan perubahan ke Supabase.

```

// app/admin/proyek/edit/[id]/page.tsx
import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';
import { createSupabaseServerClient } from '@/lib/supabase-server';

async function editProyekAction(formData: FormData) {
  'use server';

  const id = formData.get('id') as string;
  const supabase = await createSupabaseServerClient();

  const { error } = await supabase
    .from('proyek')

```

```

    .update({
      judul: formData.get('judul') as string,
      deskripsi: formData.get('deskripsi') as string,
      teknologi: formData.get('teknologi') as string,
      link: (formData.get('link') as string) || null,
    })
    .eq('id', id);

if (error) {
  console.error('Gagal mengedit proyek:', error.message);
}

revalidatePath('/admin/proyek');
revalidatePath('/proyek');
redirect('/admin/proyek');
}

export default async function EditProyekPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = await createSupabaseServerClient();

  const { data: proyek } = await supabase
    .from('proyek')
    .select('*')
    .eq('id', id)
    .single();

  if (!proyek) redirect('/admin/proyek');

  return (
    <div className="max-w-2xl">
      <h1 className="text-2xl font-bold text-slate-800 mb-6">Edit Proyek</h1>
      <div className="bg-white rounded-2xl border border-slate-200 p-6">
        <form action={editProyekAction} className="space-y-4">
          <input type="hidden" name="id" value={proyek.id} />
          <div>
            <label htmlFor="edit-judul" className="block text-sm font-medium text-slate-700 mb-1">Judul Proyek</label>
            <input id="edit-judul" name="judul" defaultValue={proyek.judul} required
              className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm
              focus:outline-none focus:ring-2 focus:ring-blue-500" />

```

```

    </div>
    <div>
      <label htmlFor="edit-teknologi" className="block text-sm font-medium text-slate-700 mb-1">Teknologi (pisah koma)</label>
      <input id="edit-teknologi" name="teknologi" defaultValue={proyek.teknologi}
        className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
    </div>
    <div>
      <label htmlFor="edit-deskripsi" className="block text-sm font-medium text-slate-700 mb-1">Deskripsi</label>
      <textarea id="edit-deskripsi" name="deskripsi"
        defaultValue={proyek.deskripsi} rows={3}
        className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
    </div>
    <div>
      <label htmlFor="edit-link" className="block text-sm font-medium text-slate-700 mb-1">Link Proyek (opsional)</label>
      <input id="edit-link" name="link" type="url" defaultValue={proyek.link ?? ''}
        className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500" />
    </div>
    <div className="flex gap-3">
      <button type="submit"
        className="bg-blue-600 text-white px-5 py-2 rounded-lg text-sm font-medium hover:bg-blue-700 transition-colors">
        Simpan Perubahan
      </button>
      <a href="/admin/proyek"
        className="bg-slate-100 text-slate-700 hover:bg-slate-200 px-5 py-2 rounded-lg text-sm font-medium transition-colors">
        Batal
      </a>
    </div>
  </form>
</div>
</div>
);
}

```

Step 10 - Membuat Halaman Konfirmasi Hapus & Server Action Delete

Buat folder `app/admin/proyek/hapus/[id]/`, lalu buat file `page.tsx`. Halaman ini menampilkan konfirmasi sebelum menghapus agar data tidak terhapus secara tidak sengaja.

```
// app/admin/proyek/hapus/[id]/page.tsx
import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';
import { createSupabaseServerClient } from '@/lib/supabase-server';

async function hapusProyekAction(formData: FormData) {
  'use server';

  const id = formData.get('id') as string;
  const supabase = await createSupabaseServerClient();

  const { error } = await supabase
    .from('proyek')
    .delete()
    .eq('id', id);

  if (error) {
    console.error('Gagal menghapus proyek:', error.message);
  }

  revalidatePath('/admin/proyek');
  revalidatePath('/proyek');
  redirect('/admin/proyek');
}

export default async function HapusProyekPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = await createSupabaseServerClient();

  const { data: proyek } = await supabase
    .from('proyek')
    .select('judul, id')
    .eq('id', id)
    .single();

  if (!proyek) redirect('/admin/proyek');
```

```

return (
  <div className="max-w-md">
    <h1 className="text-2xl font-bold text-slate-800 mb-6">Konfirmasi Hapus</h1>
    <div className="bg-white rounded-2xl border border-red-200 p-6">
      <p className="text-slate-700 mb-2">Apakah kamu yakin ingin menghapus proyek
berikut?</p>
      <p className="font-semibold text-slate-900 mb-6">{proyek.judul}</p>
      <p className="text-sm text-red-600 mb-6">Tindakan ini tidak dapat
dibatalkan.</p>
      <form action={hapusProyekAction} className="flex gap-3">
        <input type="hidden" name="id" value={proyek.id} />
        <button type="submit"
          className="bg-red-600 text-white px-5 py-2 rounded-lg text-sm font-medium
hover:bg-red-700 transition-colors">
          Ya, Hapus
        </button>
        <a href="/admin/proyek"
          className="bg-slate-100 text-slate-700 hover:bg-slate-200 px-5 py-2
rounded-lg text-sm font-medium transition-colors">
          Batal
        </a>
      </form>
    </div>
  </div>
);
}

```

Step 11 - Uji Coba Alur Penuh & Verifikasi Keamanan

Setelah semua halaman admin selesai dibuat, lakukan pengujian alur penuh untuk memverifikasi bahwa semua bagian bekerja dengan benar, termasuk proteksi keamanannya.

Alur pengujian yang harus dilakukan oleh setiap siswa:

1. BUKA BROWSER INCOGNITO

Buka `http://localhost:3000/admin/proyek`

Hasil yang diharapkan: langsung diarahkan ke `/admin/login`

2. LOGIN

Di halaman `/admin/login`, isi email dan password admin dari Step 3

Hasil yang diharapkan: berhasil masuk ke `/admin/proyek`

3. TEST CREATE

Isi form tambah proyek, klik Simpan

Hasil yang diharapkan: proyek baru muncul di tabel admin DAN di halaman /proyek publik

4. TEST UPDATE

Klik Edit pada proyek yang baru ditambahkan, ubah judulnya, klik Simpan Perubahan

Hasil yang diharapkan: form terisi data lama, perubahan tersimpan dan terlihat di kedua halaman

5. TEST DELETE

Klik Hapus pada proyek yang baru diedit, konfirmasi

Hasil yang diharapkan: halaman konfirmasi muncul, proyek terhapus dari kedua halaman

6. LOGOUT

Klik tombol Logout di header admin

Hasil yang diharapkan: kembali ke /admin/login

7. VERIFIKASI PROTEKSI SETELAH LOGOUT

Coba buka /admin/proyek

Hasil yang diharapkan: diarahkan kembali ke /admin/login

VIII. TUGAS MANDIRI (DILANJUTKAN DI RUMAH)

Dashboard Admin Portfolio Pribadi Terproteksi

Lanjutkan pekerjaan hari ini di rumah menggunakan project profil-siswa yang sama. Tugas ini BUKAN membuat project baru kalian mengembangkan project yang sama yang sudah dikerjakan sejak Pertemuan 01. Pastikan semua fitur dari Pertemuan 01-03 tetap berfungsi (halaman portfolio publik, data dari Supabase) sebelum mulai mengerjakan tugas ini.

Deliverables Wajib Dikumpulkan:

1. Repository GitHub dengan update terbaru dari Pertemuan 04: struktur folder app/admin/ lengkap, file middleware.ts di root, dan dua file helper lib/supabase-server.ts dan lib/supabase-browser.ts. File .env.local TIDAK ikut di-commit.
2. Screenshot atau rekaman layar singkat yang menunjukkan alur penuh: login ke halaman admin, tambah proyek baru, lihat proyek tersebut di halaman /proyek publik, edit proyek, hapus proyek, dan logout.
3. Screenshot tab Policies di dashboard Supabase yang menampilkan 4 policy aktif pada tabel proyek: SELECT publik, INSERT authenticated, UPDATE authenticated, DELETE authenticated.
4. Tugas pengembangan mandiri: tambahkan minimal SATU fitur personalisasi ke halaman admin atau portfolio, misalnya kolom atau field tambahan di tabel dan form (contoh: tahun_dibuat, kategori, atau thumbnail_url), pesan selamat datang di halaman /admin/proyek yang menampilkan nama atau email admin yang login, atau tampilan halaman /proyek publik yang menampilkan jumlah total proyek.

5. Video penjelasan berbahasa Inggris (durasi 3-5 menit) yang menjelaskan: apa itu Server Actions dan bedanya dengan Route Handler, bagaimana middleware bekerja untuk melindungi halaman admin, dan demo singkat CRUD dari halaman admin. Upload ke YouTube dengan judul format: [Nama Lengkap] - Week 4 Next.js Kelas Industri UBIG - Server Actions & Admin Panel.

Tantangan Opsional (untuk yang lebih cepat):

- Menambahkan kolom gambar atau thumbnail ke tabel proyek menggunakan Supabase Storage untuk upload gambar, lalu menampilkan thumbnail tersebut di halaman /proyek publik dan di CardProyek.
- Menambahkan halaman /admin/proyek/tambah sebagai halaman terpisah (bukan form di bawah tabel) dengan UX yang lebih baik dan validasi input yang lebih lengkap.
- Menambahkan fitur pencarian atau filter di tabel admin (/admin/proyek) berdasarkan judul atau teknologi menggunakan URL search params.

IX. RUBRIK ASESMEN & EVALUASI

No.	Komponen Penilaian	Indikator Ketercapaian	Bobot	Skor Maks
1	Pemahaman Konsep Server Actions	Siswa dapat menjelaskan apa itu Server Actions, bedanya dengan Route Handler, dan kapan harus memilih mana, menggunakan kata-kata sendiri.	10%	100
2	Implementasi Supabase Auth & Proteksi Route	Halaman /admin/login berfungsi, middleware.ts berhasil memblokir akses ke /admin tanpa login dan mengarahkan ke halaman login, serta logout berfungsi dengan benar.	25%	100
3	Implementasi CRUD Penuh via Server Actions	Tiga operasi (tambah, edit, hapus) berfungsi melalui halaman admin menggunakan Server Actions, dengan revalidatePath yang memastikan perubahan langsung terlihat di halaman publik /proyek.	30%	100
4	Konfigurasi Policy Supabase yang Benar	Tabel proyek memiliki 4 policy yang benar (1 SELECT publik dari Pertemuan 03 tetap ada, ditambah 3 policy authenticated untuk INSERT/UPDATE/DELETE), dan tidak ada secret key di kode frontend.	15%	100
5	Kesinambungan dengan Project Sebelumnya	Semua halaman publik portfolio dari Pertemuan 01-03 tetap berfungsi. lib/supabase.ts dari Pertemuan 03 tidak dihapus. Halaman /proyek publik menampilkan data terbaru setelah operasi CRUD dari admin.	15%	100
6	Fitur Pengembangan Mandiri & Video Penjelasan	Minimal satu fitur personalisasi ditambahkan ke halaman admin atau portfolio. Video penjelasan berbahasa Inggris terupload ke YouTube dengan judul format yang benar dan menjelaskan Server Actions, middleware, dan demo CRUD.	5%	100

X. ROADMAP KELAS INDUSTRI - PERTEMUAN SELANJUTNYA

Setiap project yang dibuat di pertemuan ini adalah fondasi dari project minggu berikutnya. Berikut gambaran perjalanan belajar yang akan ditempuh:

Minggu	Tema / Project	Output	Status
1	Pengenalan & Dasar-Dasar Next.js - App Router, Routing, Navigasi Link	Website multi-halaman 'Profil Siswa' dengan navigasi Link	Selesai
2	Styling Tailwind, Komponen Reusable, Interaktivitas & Dynamic Routing	Website Profil & Portofolio Siswa terintegrasi dengan dynamic route [id] dan filter	Selesai
3	Pengenalan Supabase & Integrasi Database Cloud ke Website Portfolio	Data proyek portofolio tersimpan dan diambil langsung dari database Supabase, menggantikan data statis	Selesai
4	Server Actions, Form Mutasi CRUD Penuh, & Proteksi Halaman Admin	Halaman dashboard admin terproteksi untuk menambah, mengubah, dan menghapus data proyek portofolio	Pertemuan ini
5	Deployment Produksi ke Vercel & Optimasi SEO Metadata	Website live di internet dengan domain custom dan performa skor tinggi	Berikutnya

XI. DAFTAR REFERENSI

Seluruh materi dan panduan praktik dalam modul ini disusun berdasarkan dokumentasi resmi dan standar industri teknologi terkini:

- Next.js Official Documentation - Server Actions and Mutations: <https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations>
- Next.js Official Documentation - Middleware: <https://nextjs.org/docs/app/building-your-application/routing/middleware>
- Supabase Official Documentation - Auth Server-Side Rendering with Next.js: <https://supabase.com/docs/guides/auth/server-side/nextjs>
- Supabase Official Documentation - Auth Email/Password: <https://supabase.com/docs/guides/auth/passwords>
- Next.js Official Documentation - revalidatePath: <https://nextjs.org/docs/app/api-reference/functions/revalidatePath>

Mengetahui,
Guru Instruktur Sekolah

Disetujui Oleh,
Mentor Praktisi Industri

(_____)
NIP / ID:

(_____)
NIP / ID: